

InstaPilot AI Environment Key Setup Guide

A complete, step-by-step walkthrough for obtaining every credential in `.env` — including full coverage of the Instagram Graph API token chain and the YouTube OAuth refresh-token flow.

AUTHOR

Yati Bhardwaj

APPLIES TO

InstaPilot AI 1.0.0 · Next.js 16 · Prisma 5

GRAPH API VERSION

v25.0

CONTENTS

Part I — How the env contract works

- 1 · `ATTRIBUTION_ACK`
- 2 · `DATABASE_URL`
- 3 · `APP_ACCESS_KEY` & `SESSION_SECRET`

Part II — AI providers & media

- 4 · `GROK_API_KEY` (Groq)
- 5 · `GEMINI_API_KEY`
- 6 · `CEREBRAS_API_KEY`
- 7 · Cloudinary

Part III — Instagram / Meta, in full

- 8.1 Account prerequisites
- 8.2 Creating the Meta app
- 8.3 Permissions reference
- 8.4 Short-lived token
- 8.5 Long-lived exchange
- 8.6 Page ID & IG account ID
- 8.7 Page access token
- 8.8 Webhook setup
- 8.9 Verification checklist
- 8.10 Expiry & renewal

DOCUMENT VERSION

1.0 — 2 September 2026

PROJECT PATH

Insta automate\instapilot-ai

VARIABLES COVERED

38 total — 6 enforced, 32 feature-gated

8.11 Error-code reference

Part IV — YouTube, in full

- 9.1 Google Cloud project
- 9.2 OAuth consent screen
- 9.3 OAuth client ID
- 9.4 Automated token script
- 9.5 Manual fallback flow
- 9.6 Channel ID
- 9.7 Scopes & quota
- 9.8 Troubleshooting

Part V — Optional services

- 10 · Resend email
- 11 · Jamendo music
- 12 · TTS / voiceover
- 13 · Branding

Part VI — Reference

- 14 · Complete `.env` template
- 15 · Bring-up sequence
- 16 · Minimum viable sets

How the environment contract works

Before collecting a single key, understand what the application actually enforces. `.env.example` marks many variables with a , but the real gate is `scripts/check-env.mjs`, which runs automatically before `npm run dev` and `npm run db:push`. It enforces four tiers — and far fewer values than the example file implies.

The four enforcement tiers

TIER	VARIABLES	BEHAVIOUR WHEN MISSING
Prisma	DATABASE_URL	Hard fail. Must live in <code>.env</code> specifically — the Prisma CLI reads no other file.
Boot	ATTRIBUTION_ACK	Hard fail. The server refuses to start.
Login	APP_ACCESS_KEY SESSION_SECRET	Hard fail — but both are auto-generated on first run, so this only triggers if you blank them by hand.
Soft	GROK_API_KEY GEMINI_API_KEY CLOUDINARY_CLOUD_NAME CLOUDINARY_UPLOAD_PRESET	Warning only. The dashboard runs; AI features stay switched off.

Everything else is optional at boot

Meta, Resend, YouTube, Jamendo and TTS are all marked  in `.env.example` but are **not** checked by `check-env.mjs`. You can boot the app and explore the entire dashboard with **Steps 1–3 alone**. Each later section simply switches on the feature it belongs to.

The .env vs .env.local trap

This is the single most common way to lose an hour on this project. Two different tools read two different files:

TOOL	READS	CONSEQUENCE
Prisma CLI	<code>.env</code> only	Never reads <code>.env.local</code> . Ever.
Next.js runtime	<code>.env.local</code> over <code>.env</code>	Prefers <code>.env.local</code> when both exist.
Host environment	<code>process.env</code>	Wins over both files.

Why this matters

If DATABASE_URL differs between the two files, `db:push` writes the schema to one database while the running app reads another — and the app then reports missing tables with no obvious cause. `check-env.mjs` deliberately hard-fails on this mismatch rather than letting it through. **Recommendation: keep a single `.env` and delete `.env.local`.**

Notation used in this document

 blocks boot or login  warns, feature disabled  silently skipped  generated for you

1 **ATTRIBUTION_ACK**

The startup gate in `lib/attribution.ts`. No signup, no account, nothing to obtain.

The project is MIT-licensed and free to fork, re-skin and run commercially. The one condition is that credit to the original author stays visible. The server checks this value at boot. **Nothing is transmitted anywhere** — the string is compared locally.

```
ATtribution_ack="https://github.com/ys941"
```

This line already exists at the top of `.env.example`, so it carries over automatically when `.env` is seeded. Simply do not delete it.

2 DATABASE_URL REQUIRED

PostgreSQL connection string. Must be present in `.env`, not `.env.local`.

Option A — Docker (recommended)

The default value already matches the credentials baked into `docker-compose.yml`, so it works with zero editing:

```
DATABASE_URL=postgresql://instapilot:instapilot_secret@localhost:5432/instapilot_db
```

```
docker compose up -d postgres
```

Port 5432 contention on this machine

Three other projects — **youtubepilot-ai**, **cardioflow-ai whitelabel** and **cardioflow-ai-from-onedrive** — each publish their own Postgres container on port 5432. If a sibling is already running, InstaPilot connects to the *wrong database* and reports missing tables. Stop the sibling container first, or use `start-all.bat`, which detects and resolves the conflict automatically.

Option B — Managed Postgres (Neon, Supabase, Railway)

Copy the connection string from the provider dashboard. Append `?sslmode=require` if the provider does not already include it:

```
DATABASE_URL=postgresql://user:pass@ep-xyz.eu-central-1.aws.neon.tech/instapilot?sslmode=require
```

3 APP_ACCESS_KEY & SESSION_SECRET AUTO-GENERATED

Do **not** invent these yourself. Leave both blank and let the app generate them.

On the first run of `npm run dev`, `npm run db:push` or `start-all.bat`, `check-env.mjs` creates `.env` from `.env.example` and fills in the two values a human has no meaningful way to choose:

VARIABLE	GENERATED AS	PURPOSE
SESSION_SECRET	randomBytes(32).hex → 64 chars	HMAC secret signing the auth session cookie
APP_ACCESS_KEY	randomBytes(9).base64url → 12 chars	The key you type at the dashboard login screen

Watch your terminal on that first run — the login key is printed once:

```
i No .env file was found, so one has been created from .env.example.
```

Your dashboard login key has been generated for you:

```
xK3mQ9pL2vNw
```

It is saved in .env as APP_ACCESS_KEY - change it there any time.

Both values are ordinary random strings, so you may replace them with anything you prefer. To regenerate a session secret by hand:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Checkpoint — the app now runs

With Steps 1–3 complete you can start the server and log in. Everything that follows enables a specific feature and can be added incrementally, in any order, at any time.

AI providers & media hosting

Four keys switch on the generation engine. All three AI providers have usable free tiers and none require a payment card. The application routes every AI call through `lib/ai-factory.ts`, which resolves an ordered **provider · model** chain per task lane — **content**, **reply** and **vision** — so one provider's outage or quota wall falls through to the next instead of failing the day.

4 GROK_API_KEY SOFT-REQUIRED

This is **Groq** (LPU inference), not xAI's Grok. The variable name is historical.

Powers comment replies, DM replies, Whisper transcription for inbound voice notes, text-to-speech, and content generation fallback.

- 1 Go to **console.groq.com** and sign in with Google or GitHub.
- 2 Open **API Keys** in the left sidebar.
- 3 Click **Create API Key**, name it `instapilot`, then **Submit**.
- 4 Copy the key **immediately** — it is displayed exactly once.

```
GROK_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Related variables — leave all three blank

GROK_API_URL defaults to the Groq OpenAI-compatible endpoint. AI_MODEL_MAIN defaults to `llama-3.3-70b-versatile`. AI_MODEL_FAST defaults to `llama-3.1-8b-instant`. The code accepts both GROK_API_KEY and GROQ_API_KEY spellings.

5 GEMINI_API_KEY SOFT-REQUIRED

Google AI Studio key. Powers content generation, vision analysis and structured JSON output.

The vision lane uses this to analyse rendered cards — it drives caption generation, hashtag enrichment and background-music mood selection.

- 1 Go to **aistudio.google.com/apikey**.
- 2 Click **Create API key**. Select an existing Google Cloud project, or let AI Studio create one for you.
- 3 Copy the generated string.

```
GEMINI_API_KEY=AIzaSyXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Free tier, no card required. Leave `GEMINI_MODEL` blank — it defaults to `gemini-2.5-flash`, and the per-task chain is configured in the app under **Settings** → **AI Config** rather than in `.env`.

6 CEREBRAS_API_KEY OPTIONAL

The middle link in the fallback chains. Skip unless you want three-deep redundancy.

- 1 Sign up at **cloud.cerebras.ai**.

- 2 Open **API Keys** and generate one.

```
CEREBRAS_API_KEY=csk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

This key can live in the database instead

Cerebras and Gemini are the two providers flagged `dbKey: true` in `lib/aiModels.ts`, meaning they can be pasted into **Settings** → **AI Config** in the running app rather than into `.env`. Groq is environment-only.

7 Cloudinary SOFT-REQUIRED

Media hosting. Without it, Instagram publishing cannot work at all.

Why this is not optional for Instagram

The Instagram Graph API cannot accept raw image or video bytes. It requires a **publicly reachable HTTPS URL** that Meta's servers fetch themselves. Cloudinary is that host. The rendered cards and Short MP4s are uploaded there first, then the resulting URL is handed to Meta.

7a — Cloud name

- 1 Sign up at **cloudinary.com** (free tier includes 25 GB storage and bandwidth).
- 2 On the Dashboard, find **Product Environment Credentials** and copy the **Cloud Name**.

```
CLOUDINARY_CLOUD_NAME=dxxxxxxx
```

7b — Unsigned upload preset

- 1 Click the **gear icon** (Settings) → **Upload** tab.
- 2 Scroll to **Upload presets** → **Add upload preset**.
- 3 Set **Signing Mode** to **Unsigned**. This is the critical setting — a signed preset will fail with a 401 at upload time.
- 4 Save, then copy the preset name.

```
CLOUDINARY_UPLOAD_PRESET=instapilot_unsigned
```

7c — Signed credentials OPTIONAL

Only needed so the app can *delete* media after a successful publish, keeping your free-tier storage clear. Both values are on the same dashboard panel as the cloud name.

```
CLOUDINARY_API_KEY=123456789012345  
CLOUDINARY_API_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Allowlisted by the SSRF guard

`lib/urlSafety.ts` explicitly allows `res.cloudinary.com`. If you substitute a different media host, add it to that allowlist or outbound fetches to it will be rejected.

Instagram / Meta — the full token chain

Seven variables, and by far the most involved part of the setup. The difficulty is not the app configuration — it is that Meta requires a specific **account topology** before any token will carry the permissions you need. Work through 8.1 to 8.8 in order; skipping ahead produces tokens that look valid but silently lack scope.

What you are building

VARIABLE	TYPE	USED FOR
FACEBOOK_APP_ID	App identifier	OAuth, webhook registration
FACEBOOK_APP_SECRET	App secret	Token exchange and webhook HMAC verification
INSTAGRAM_ACCESS_TOKEN	Long-lived user token	Publishing, insights, comments
INSTAGRAM_BUSINESS_ACCOUNT_ID	Numeric ID	Target account for all publish calls
FACEBOOK_PAGE_ID	Numeric ID	DM conversations endpoint
FACEBOOK_PAGE_ACCESS_TOKEN	Page token	Sending and reading DMs
INSTAGRAM_USERNAME	Handle	Self-reply suppression (loop prevention)
WEBHOOK_VERIFY_TOKEN	Self-chosen string	Webhook handshake with Meta

8.1 Account prerequisites

Do this first, in the Instagram mobile app. Nothing else will work until it is done.

- 1 **Convert Instagram to a Business or Creator account.** A personal account cannot publish through the API. Instagram app → **Settings and privacy** → **Account type and tools** → **Switch to professional account** → choose **Business**.
- 2 **Create a Facebook Page** if you do not have one, at facebook.com/pages/create. It can be minimal — a name and category are enough.
- 3 **Link the Page to the Instagram account.** Instagram app → **Settings** → **Accounts Centre** → **Connected experiences** → **Accounts** → add the Facebook Page. Or from the Page: **Settings** → **Linked accounts** → **Instagram** → **Connect**.
- 4 **Verify the link.** On the Facebook Page, **Settings** → **Linked accounts** should show the Instagram handle. If it does not, no token will ever return an `instagram_business_account` field, and Step 8.6 will fail.

The most common failure in this entire guide

If the Page ↔ Instagram link is missing or was made through a personal account, the Graph API returns a Page object with **no** `instagram_business_account` field. There is no error message — the field is simply absent. If Step 8.6 returns an empty object, come back and redo this section.

8.2 Creating the Meta app

- 1 Go to developers.facebook.com and register as a developer if prompted.
- 2 **My Apps** → **Create App**.

- 3 Select use case **Other**, then app type **Business**.
- 4 Give it a name, select your Business portfolio if asked, and create.
- 5 In the left sidebar open **App settings** → **Basic** and copy both values. Click **Show** to reveal the secret (you will re-enter your password).

```
FACEBOOK_APP_ID=1234567890123456
FACEBOOK_APP_SECRET=a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6
```

- 6 Scroll down and click **Add Product**. Add **Instagram Graph API** and **Facebook Login for Business**.

FACEBOOK_APP_SECRET does double duty

Besides token exchange, this value verifies the X-Hub-Signature-256 HMAC on every inbound webhook in `app/api/webhook/route.ts`. A wrong or missing secret means webhook events are silently rejected, and comment/DM replies fall back to the slower five-minute polling path.

8.3 Permissions reference

Request all nine. Missing one produces a token that works for some calls and returns error 200 or 10 for others.

PERMISSION	WHAT BREAKS WITHOUT IT
instagram_basic	Everything — baseline account read
instagram_content_publish	Cannot publish posts, carousels, Reels or Stories
instagram_manage_comments	Cannot read or reply to comments
instagram_manage_messages	Cannot read or send DMs
instagram_manage_insights	Analytics page shows zeros
pages_show_list	<code>/me/accounts</code> returns nothing — cannot find your Page ID
pages_read_engagement	Page-scoped conversation reads fail
pages_manage_metadata	Webhook binding silently fails to stick
business_management	Business-portfolio-owned assets are invisible

8.4 Generating the short-lived token

- 1 Open the Graph API Explorer at developers.facebook.com/tools/explorer.
- 2 In the top-right **Meta App** dropdown, select the app you created in 8.2.
- 3 Set **User or Page** to **User token**.
- 4 Click **Add a Permission** and tick all nine permissions from the table in 8.3.
- 5 Click **Generate Access Token**. A Facebook dialog opens — accept it, and make sure the correct Page and Instagram account are ticked in the asset selection screens.
- 6 Copy the token that appears in the **Access Token** field.

This token expires in about one hour

It is deliberately short-lived. Do not put it in `.env` — continue straight to 8.5 and exchange it first.

8.5 Exchanging for a long-lived token

Converts the 1-hour token into a 60-day token. Run this in any terminal.

```
curl -s "https://graph.facebook.com/v25.0/oauth/access_token\
?grant_type=fb_exchange_token\
&client_id=YOUR_APP_ID\
&client_secret=YOUR_APP_SECRET\
&fb_exchange_token=SHORT_LIVED_TOKEN_FROM_8.4"
```

The response is JSON:

```
{
  "access_token": "EAAxxxxxxxxxxxxxxxxxx...",
  "token_type": "bearer",
  "expires_in": 5183944
}
```

`expires_in` is seconds — 5,183,944 is approximately 60 days. Save the token:

```
INSTAGRAM_ACCESS_TOKEN=EAAxxxxxxxxxxxxxxxxxx...
```

PowerShell alternative

If `curl` is unavailable, use:

```
Invoke-RestMethod "https://graph.facebook.com/v25.0/oauth/access_token?grant_type=fb_exchange_token&
client_id=APP_ID&client_secret=APP_SECRET&fb_exchange_token=SHORT_TOKEN"
```

8.6 Finding the Page ID and Instagram account ID

Step one — list your Pages

```
curl -s "https://graph.facebook.com/v25.0/me/accounts?access_token=LONG_LIVED_TOKEN"
```

Response:

```
{
  "data": [
    {
      "access_token": "EAAyyyyyyyyyyyyyy...",
      "category": "Health/Beauty",
      "name": "Your Page Name",
      "id": "100234567890123"
    }
  ]
}
```

Take the `id` field:

```
FACEBOOK_PAGE_ID=100234567890123
```

Keep this response open

The `access_token` field in the same object is your **Page access token** — needed in 8.7. Do not close the terminal yet.

Step two — resolve the linked Instagram account

```
curl -s "https://graph.facebook.com/v25.0/PAGE_ID\
?fields=instagram_business_account\
&access_token=LONG_LIVED_TOKEN"
```

```
{
  "instagram_business_account": { "id": "17841400000000000" },
  "id": "100234567890123"
}
```

```
INSTAGRAM_BUSINESS_ACCOUNT_ID=17841400000000000
INSTAGRAM_USERNAME=yourhandle
```

`INSTAGRAM_USERNAME` is your handle without the `@`. It is used to detect and skip the bot's own comments — without it, the auto-reply engine can reply to itself in a loop.

If `instagram_business_account` is absent

The response will contain only `{"id": "..."}.` This always means the Page ↔ Instagram link from Step 8.1 is incomplete, or the token lacks `instagram_basic`. Return to 8.1 and verify the link on the Page itself.

8.7 The Page access token

Required for direct messages. This is a *different* token from the user token in 8.5.

Take the `access_token` value from the `/me/accounts` response in 8.6 — the one nested inside your Page's object:

```
FACEBOOK_PAGE_ACCESS_TOKEN=EAayyyyyyyyyyyy...
```

If you closed that terminal, fetch it directly:

```
curl -s "https://graph.facebook.com/v25.0/PAGE_ID\
?fields=access_token&access_token=LONG_LIVED_TOKEN"
```

Page tokens must carry `pages_manage_metadata`

A Page token derived from a user token that lacked this scope will appear to work for reads but fails to bind the webhook — events never arrive and there is no error. If DMs are not being received, re-run 8.4 with all nine permissions and derive a fresh Page token.

How the app resolves this token

`getPageToken()` in `lib/catchup.ts` tries four sources in order: the `FACEBOOK_PAGE_ACCESS_TOKEN` env var (primary brand only), then `/me/accounts`, then `/pageId?fields=access_token`, and finally falls back to the user token. Setting the env var explicitly is the fastest and most reliable path.

8.8 Webhook setup

Enables real-time comment and DM delivery instead of five-minute polling.

Generate the verify token — you choose this one

It is an arbitrary shared secret between you and Meta. Generate something unguessable:

```
node -e "console.log(require('crypto').randomBytes(16).toString('hex'))"
```

```
WEBHOOK_VERIFY_TOKEN=a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6
```

Register the callback with Meta

- 1 Your app must be reachable over public HTTPS. For local development, tunnel it:

```
npx localtunnel --port 3000
# or
ngrok http 3000
```

- 2 In the App Dashboard, open **Instagram** → **Webhooks** (or **Webhooks** in the sidebar, then select **Instagram**).
- 3 Click **Subscribe to this object** and enter:
 - **Callback URL:** `https://your-domain.com/api/webhook`
 - **Verify Token:** the exact string you generated above
- 4 Click **Verify and Save**. Meta immediately sends a `GET` to your callback with a challenge; the route echoes it back if the tokens match. Your server must already be running.
- 5 Subscribe to the fields **comments** and **messages**.

Polling still works without a webhook

The catchup loop falls back to a five-minute comment poll whenever the webhook is silent. Real-time delivery is a latency improvement, not a hard requirement — you can defer this section and add it later.

8.9 Verification checklist

Run these four calls before starting the app. Each isolates one failure mode.

CHECK	COMMAND AND EXPECTED RESULT
Token is valid	<code>GET /v25.0/me?access_token=TOKEN</code> Returns your name and id, not an OAuthException.
Token has scopes	<code>GET /v25.0/me/permissions?access_token=TOKEN</code> All nine listed with <code>status: granted</code> .
IG account resolves	<code>GET /v25.0/IG_ACCOUNT_ID?fields=username,followers_count&access_token=TOKEN</code> Returns your handle and follower count.
Page token works	<code>GET /v25.0/PAGE_ID/conversations?platform=instagram&access_token=PAGE_TOKEN</code> Returns a data array — or error 3, see below.

Token debugger

For any token, developers.facebook.com/tools/debug/accesstoken shows its type, expiry, granted scopes and which app issued it. This is the fastest way to diagnose a token that "should work".

8.10 Expiry and renewal

TOKEN	LIFETIME	RENEWAL
Short-lived user token	~1 hour	Never used directly — exchange immediately
INSTAGRAM_ACCESS_TOKEN	60 days	Re-run 8.4 and 8.5, or refresh before expiry
FACEBOOK_PAGE_ACCESS_TOKEN	Does not expire [*]	Re-derive if the parent user token is revoked

^{*} A Page token derived from a *long-lived* user token is itself long-lived and in practice does not expire — but it is invalidated if you change your password, revoke the app, or the parent user token is invalidated.

To refresh the user token before it expires, exchange the current one again:

```
curl -s "https://graph.facebook.com/v25.0/oauth/access_token\
?grant_type=fb_exchange_token\
&client_id=APP_ID&client_secret=APP_SECRET\
&fb_exchange_token=CURRENT_LONG_LIVED_TOKEN"
```

Set a calendar reminder for day 55

There is no automatic renewal in the codebase. When the token expires, publishing fails with error 190 and the daily health email reports the failure — but nothing self-heals. A recurring reminder is the practical fix.

8.11 Meta error-code reference

These are the codes the application explicitly handles, and what each one actually means.

CODE	MEANING	FIX
3	App in Development Mode — Conversations API blocked	Expected before App Review. The app detects this and replays webhook-delivered DMs from <code>ActivityLog</code> instead. No action needed.
190	Invalid or expired access token	Token expired at 60 days, or the password changed. Re-run 8.4 → 8.5.
200	Permission denied — missing scope	The token lacks one of the nine permissions. Re-generate with all of them.
10	Permission denied for this action	Treated as a permanent DM failure by <code>isPermanentDmError()</code> — the message is not retried.
551	User unavailable / cannot receive message	Also treated as permanent. The recipient blocked the account or deleted it.
100 / sub 33	Object does not exist, or no permission to view it	Usually a wrong ID. When it occurs during container polling, the app skips polling and waits a fixed delay instead.
sub 2534022	Outside the allowed messaging window	Meta's 24-hour reply window elapsed. Permanent — not retried.
4 / 17	Application or user request limit reached	The app calls <code>markRateLimited()</code> , pauses all Meta calls for one hour, and sends a rate-limit alert email.

Development Mode is not a blocker

Until the app passes App Review, the Conversations API returns error 3 and DM *fetching* is walled off. The system still works: webhook-delivered DMs are logged to **ActivityLog** as **DM_RECEIVED** and replayed by the catchup loop, which replies to any without a matching **DM_AUTO_REPLIED** entry within 48 hours. Publishing, comments and insights are unaffected.

YouTube — OAuth and the refresh token

Four variables. Unlike Meta, the hard part here is automated: the repository ships `scripts/youtube-auth.mjs`, which opens the consent screen, catches the redirect on a local server and prints the refresh token. Your manual work is limited to creating the Google Cloud project and OAuth client.

VARIABLE	SOURCE	PURPOSE
YOUTUBE_CLIENT_ID	Google Cloud	OAuth client identity
YOUTUBE_CLIENT_SECRET	Google Cloud	OAuth client secret
YOUTUBE_REFRESH_TOKEN	Auth script	Long-lived channel authorisation
YOUTUBE_CHANNEL_ID	YouTube Studio	Display only

9.1 Google Cloud project and APIs

- 1 Go to **console.cloud.google.com** and sign in with the Google account that **owns the YouTube channel**.
- 2 Click the project dropdown in the top bar → **New Project**. Name it `instapilot` and create it. Make sure it is selected afterwards.
- 3 Open **APIs & Services** → **Library** and enable both of these:
 - **YouTube Data API v3** — uploads, comments, video metadata
 - **YouTube Analytics API** — time-series channel statistics

Search each by name and click **Enable**.

Use the channel-owning Google account

If your channel is a Brand Account, the owning Google account is the one listed under YouTube Studio → Settings → Permissions as Owner. Authorising with a different account produces a token that uploads to the wrong channel — or to no channel at all.

9.2 OAuth consent screen

- 1 Open **APIs & Services** → **OAuth consent screen**.
- 2 Select **External** as the user type, then **Create**.
- 3 Fill the required fields only: **App name**, **User support email**, **Developer contact email**. Everything else can stay blank.
- 4 On the **Scopes** screen, click **Save and Continue** without adding anything — the script requests scopes at runtime.
- 5 On the **Test users** screen, click **Add Users** and add **your own Google address**. This step is mandatory.
- 6 Review and finish. Leave the app in **Testing** status.

Two consequences of Testing status

One: if your account is not in the Test users list, consent fails with *"Access blocked: has not completed the Google verification process"*.

Two: refresh tokens issued by an app in Testing status **expire after seven days**. For continuous unattended publishing you must eventually click **Publish App** on the consent screen to move it to Production. For a personal channel using only your own data, Google does not require a full verification review to do this.

9.3 Creating the OAuth client ID

- 1 Open **APIs & Services** → **Credentials**.
- 2 **Create Credentials** → **OAuth client ID**.
- 3 Application type: **Desktop app**. Name it `instapilot-cli`.

Desktop app is correct because `youtube-auth.mjs` runs a temporary local HTTP server to catch the redirect. If you choose **Web application** instead, you must manually add `http://localhost:4773` under **Authorized redirect URIs** or the flow fails with `redirect_uri_mismatch`.

- 4 Click **Create**. Copy both values from the dialog:

```
YOUTUBE_CLIENT_ID=123456789012-abcdefg.apps.googleusercontent.com
YOUTUBE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxxxxxx
```

- 5 Paste both into `.env` before continuing — the script reads them from there.

9.4 Generating the refresh token

One command. The script handles the browser, the redirect and the code exchange.

```
npm run youtube:auth
```

What happens:

- 1 The script starts a temporary HTTP server on **port 4773**.
- 2 It opens your default browser at the Google consent screen. If the browser does not open, the URL is printed — paste it manually.
- 3 Sign in with the channel-owning account and approve the four scopes.
- 4 Google redirects to `http://localhost:4773`, the script catches the code and exchanges it.
- 5 The tokens are printed to your terminal:

```
✓ Refresh token obtained.

YOUTUBE_CLIENT_ID=123456789012-abcdefg.apps.googleusercontent.com
YOUTUBE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxxxxxx
YOUTUBE_REFRESH_TOKEN=1//0gxxxxxxxxxxxxxxxxxxxxxxxxxx
```

The script's output says ".env.local" — put it in .env instead

The printed instruction reads "Add these to Railway / .env.local". Follow the project's current convention and use `.env`. Splitting configuration across both files is the mismatch trap described in Part I.

To run the script with a different port, or without editing `.env` first:

```
node scripts/youtube-auth.mjs --id=CLIENT_ID --secret=CLIENT_SECRET --port=4773
```

Why the token is reliable

The script sets `access_type: "offline"` and `prompt: "consent"`, which forces Google to return a **fresh refresh token on every run**. Google normally issues a refresh token only on first authorisation, so re-running the script after revoking access is always safe.

9.5 Manual fallback flow

Use this only if the script cannot run — for example on a headless server with no browser.

Step one — build the consent URL

Replace `CLIENT_ID` and open the result in any browser:

```
https://accounts.google.com/o/oauth2/v2/auth
?client_id=CLIENT_ID
&redirect_uri=http://localhost:4773
&response_type=code
&access_type=offline
&prompt=consent
&scope=https://www.googleapis.com/auth/youtube.upload%20https://www.googleapis.com/auth/youtube.readonly%20https://www.googleapis.com/auth/youtube.force-ssl%20https://www.googleapis.com/auth/yt-analytics.readonly
```

Step two — capture the code

After approving, the browser is redirected to `http://localhost:4773/?code=4/0AXXXXXX&scope=...`. The page will fail to load — that is expected. Copy the `code` parameter from the address bar.

Step three — exchange it for a refresh token

```
curl -s -X POST https://oauth2.googleapis.com/token \
-d "code=PASTE_CODE_HERE" \
-d "client_id=CLIENT_ID" \
-d "client_secret=CLIENT_SECRET" \
-d "redirect_uri=http://localhost:4773" \
-d "grant_type=authorization_code"
```

```
{
  "access_token": "ya29.a0AfH6...",
  "expires_in": 3599,
  "refresh_token": "1//0gxxxxxxxxxxxxx",
  "scope": "https://www.googleapis.com/auth/youtube.upload ...",
  "token_type": "Bearer"
}
```

```
YOUTUBE_REFRESH_TOKEN=1//0gxxxxxxxxxxxxx
```

The authorisation code is single-use and expires in minutes

If the exchange returns `invalid_grant`, the code was already used or has expired. Repeat from step one to obtain a fresh code. Also confirm the `redirect_uri` in the exchange matches the one in the consent URL **exactly** — including port and the absence of a trailing slash.

9.6 Channel ID

Display only — the app functions without it.

Fastest route

YouTube Studio → **Settings** → **Channel** → **Advanced settings**. The channel ID is shown under *Channel ID* and always begins with `UC`.

```
YOUTUBE_CHANNEL_ID=UCbVZPr6jApgMGB_p28s9_6g
```

Via the API

Using an access token obtained from the refresh token:

```
curl -s "https://www.googleapis.com/youtube/v3/channels?part=id,snippet&mine=true" \
-H "Authorization: Bearer ACCESS_TOKEN"
```

9.7 Scopes and quota

The four scopes the script requests

SCOPE	GRANTS
youtube.upload	Uploading videos and Shorts
youtube.readonly	Reading channel and video statistics
youtube.force-ssl	Reading and replying to comments, setting thumbnails
yt-analytics.readonly	Detailed time-series analytics

Quota — the real constraint

The YouTube Data API grants **10,000 quota units per day** by default. Costs are not uniform:

OPERATION	UNITS	PER DAY AT DEFAULT QUOTA
Video upload	1,600	~6 uploads
Set thumbnail	50	—
Insert comment	50	—
List videos / channels	1	—

Roughly six uploads per day is the practical ceiling

A single Short costs about 1,700 units once the custom thumbnail and the seeded engagement comment are included. At 10,000 units that is five to six Shorts daily — comfortably above a normal schedule, but worth knowing before you configure an aggressive per-weekday plan. Additional quota can be requested through the Google Cloud console, though approval is not guaranteed.

9.8 YouTube troubleshooting

SYMPTOM	CAUSE AND FIX
redirect_uri_mismatch	The OAuth client is type Web application . Either recreate it as Desktop app , or add <code>http://localhost:4773</code> to Authorized redirect URIs.
invalid_grant	The refresh token was revoked, the seven-day Testing-status expiry elapsed, or a manual auth code was reused. Re-run <code>npm run youtube:auth</code> .
Access blocked — verification	Your Google account is not in the OAuth consent screen's Test users list. Add it.
quotaExceeded	Daily 10,000 units exhausted. Resets at midnight Pacific Time. Reduce scheduled uploads or request more quota.
Uploads to the wrong channel	You authorised with an account that owns a different channel. Revoke access at myaccount.google.com/permissions and re-run the script with the correct account.
Token dies every 7 days	The consent screen is still in Testing . Click Publish App to move it to Production.
accessNotConfigured	YouTube Data API v3 was never enabled for the project, or the wrong project is selected. Re-check Step 9.1.

Optional services

None of these block anything. Each is skipped silently when its key is absent — the relevant feature simply does not run.

10 Resend — email reports OPTIONAL

Powers the daily health email, morning digest and failure alerts in `lib/notifier.ts`.

- 1 Sign up at **resend.com**. The free tier covers 3,000 emails per month.
- 2 Open **API Keys** → **Create API Key**. Permission: **Sending access**.

```
RESEND_API_KEY=re_XXXXXXXXXXXXXXXXXXXXX
NOTIFICATION_EMAIL=you@example.com
```

- 3 Leave `RESEND_FROM` blank to use `onboarding@resend.dev`, which works immediately with no DNS setup. To send from your own domain, verify it under **Domains** first, then:

```
RESEND_FROM=InstaPilot <alerts@yourdomain.com>
```

`DM_AUTO_REPLY` is an unrelated optional value in the same block: a plain-text fallback used only when every AI provider in the reply chain is unreachable.

Verify before relying on it

Once the app is running, use **Settings** → **Notifications** → **Send test email**, or POST `/api/test-email`, to confirm delivery end to end.

11 Jamendo — background music OPTIONAL

Free Creative Commons tracks, selected by the vision model to match each card's mood.

- 1 Register at **devportal.jamendo.com**.
- 2 Create an application and copy the **Client ID** — not the secret.

```
JAMENDO_CLIENT_ID=abc12345
```

When absent, `lib/music.ts` reports music as unconfigured and Shorts render silently with no soundtrack. No error is raised.

12 TTS / voiceover OPTIONAL

All six variables have working defaults. Leave them blank unless self-hosting.

```
TTS_PROVIDER=groq           # "groq" (default) or "canopy"
GROQ_TTS_MODEL=             # default: canopylabs/orpheus-v1-english
GROQ_TTS_VOICE=            # default: autumn – per-brand override in Settings
CANOPY_TTS_URL=            # only for a self-hosted Orpheus endpoint
CANOPY_TTS_KEY=            # bearer key for the above
CANOPY_TTS_VOICE=         # default: tara
```

The default path reuses `GROK_API_KEY` — no additional key is needed for voiceover.

Groq TTS requires terms acceptance before first use

Groq's Orpheus TTS models are gated behind an organisation-admin terms acceptance in the Groq console. Until accepted, the first synthesis call returns HTTP 403 with no other explanation. Accept the model terms at console.groq.com before enabling voiceover. This is the same gate encountered on the CardioFlow project.

Voiceover and burned captions are opt-in for a reason

Each adds a TTS call, a Whisper call and a full re-encode per Short. On a two-core CPU-only machine this is the slowest part of the pipeline by a wide margin.

13 Branding OPTIONAL

Fallbacks only. The real white-label configuration lives in the database, not here.

```
BRAND_NAME=Your Brand AI
NEXT_PUBLIC_APP_NAME=Your Brand AI
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Configure the brand in the app, not in .env

App name, niche, persona, handles, colours, content-type labels, topics and prompts are all set per-brand under **Settings** → **Brand**. Better still, open **Settings** → **AI Setup** and describe your brand in plain English — it generates the skin, content types, topics, schedule and persona for you. These environment variables are only fallbacks used before any brand exists.

Reference

14 Complete .env template

Every variable in one place, in the order the application resolves them.

```

# — Attribution (required - server will not boot without it) —————
ATTRIBUTEION_ACK="https://github.com/ys941"

# — Branding (white-label fallbacks; real config is in Settings) —————
BRAND_NAME=
NEXT_PUBLIC_APP_NAME=
NEXT_PUBLIC_APP_URL=http://localhost:3000

# — Core (required) —————
DATABASE_URL=postgresql://instapilot:instapilot_secret@localhost:5432/instapilot_db
APP_ACCESS_KEY=                # auto-generated on first run - printed in terminal
SESSION_SECRET=                # auto-generated on first run

# — AI providers —————
GROK_API_KEY=                  # gsk...      console.groq.com
GROK_API_URL=                  # blank = default
AI_MODEL_MAIN=                 # blank = llama-3.3-70b-versatile
AI_MODEL_FAST=                 # blank = llama-3.1-8b-instant
GEMINI_API_KEY=                # AIza...   aistudio.google.com/apikey
GEMINI_MODEL=                  # blank = gemini-2.5-flash
CEREBRAS_API_KEY=              # csk...    cloud.cerebras.ai (optional)

# — AI voiceover / TTS (opt-in) —————
TTS_PROVIDER=
GROQ_TTS_MODEL=
GROQ_TTS_VOICE=
CANOPY_TTS_URL=
CANOPY_TTS_KEY=
CANOPY_TTS_VOICE=

# — Instagram / Meta —————
INSTAGRAM_ACCESS_TOKEN=        # EAA...   long-lived, 60 days
INSTAGRAM_BUSINESS_ACCOUNT_ID= # 1784...
INSTAGRAM_USERNAME=            # handle, no @
FACEBOOK_PAGE_ACCESS_TOKEN=    # EAA...   Page token, required for DMs
FACEBOOK_PAGE_ID=              # 1002...
FACEBOOK_APP_ID=
FACEBOOK_APP_SECRET=           # also verifies webhook HMAC
WEBHOOK_VERIFY_TOKEN=          # any random string you choose

# — Media hosting —————
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_UPLOAD_PRESET=      # must be UNSIGNED
CLOUDINARY_API_KEY=            # optional - for post-publish deletion
CLOUDINARY_API_SECRET=

# — Email —————
RESEND_API_KEY=                # re...
RESEND_FROM=                   # blank = onboarding@resend.dev
NOTIFICATION_EMAIL=
DM_AUTO_REPLY=

# — YouTube —————
YOUTUBE_CLIENT_ID=             # ....apps.googleusercontent.com
YOUTUBE_CLIENT_SECRET=         # GOCSPX-...
YOUTUBE_REFRESH_TOKEN=         # 1//0g... from: npm run youtube:auth
YOUTUBE_CHANNEL_ID=            # UC...    display only

# — Music for Shorts —————
JAMENDO_CLIENT_ID=

```

15 Bring-up sequence

```
# 1. Install. This also runs prisma generate via postinstall.
npm install

# 2. Let check-env seed .env and print your login key.
# (npm run dev triggers it too - this is just to see the key early.)
npm run check-env

# 3. Start the database.
docker compose up -d postgres

# 4. Fill in keys from Parts II-V, then push the schema.
npm run db:generate
npm run db:push

# 5. Run.
npm run dev                # http://localhost:3000
```

On Windows, one command does all five
`start-all.bat` runs the environment check, brings up Postgres and Redis, resolves the port-5432 conflict with sibling projects, runs Prisma, and starts the dev server.

First login

- 1 Open `http://localhost:3000`.
- 2 Enter the `APP_ACCESS_KEY` printed during Step 2.
- 3 Open **Settings** → **AI Setup** and describe your brand in plain English. It generates the skin, content types, topics, schedule and persona.
- 4 Review **Settings** → **Auto-Post** before enabling auto-publish.

16 Minimum viable sets

Add only what the feature you want actually requires.

GOAL	SECTIONS NEEDED	KEYS
Boot the dashboard and explore	1, 2, 3	3 (2 auto)
Generate content with AI, no publishing	+ 4, 5	5
Publish to Instagram	+ 7, 8	15
Publish Shorts to YouTube	+ 9	19
Email reports and alerts	+ 10	21
Voiceover and background music	+ 11, 12	22

Troubleshooting index

SYMPTOM	SEE
Prisma P1012 — DATABASE_URL not found	Part I, the <code>.env</code> vs <code>.env.local</code> trap
App reports missing tables after db:push	Part I — schema pushed to a different database
Connects to the wrong database entirely	Section 2 — port 5432 contention
Login returns HTTP 500	Section 3 — APP_ACCESS_KEY or SESSION_SECRET blanked
Cloudinary upload returns 401	Section 7b — preset is signed, must be unsigned
<code>instagram_business_account</code> missing	Section 8.1 — Page ↔ Instagram link incomplete
DMs not arriving	Section 8.7 and 8.11 — Page token scope, or error 3
Publishing stops after two months	Section 8.10 — 60-day token expiry
Webhook events never arrive	Section 8.2 — wrong APP_SECRET breaks HMAC
YouTube <code>redirect_uri_mismatch</code>	Section 9.3 — use Desktop app client type
YouTube token dies weekly	Section 9.2 — consent screen still in Testing
TTS returns HTTP 403	Section 12 — Groq model terms not accepted

A note on operational safety

You are responsible for what this publishes

Both Meta and YouTube enforce automation policies, and this system publishes to real accounts unattended. Read those policies, set the schedule conservatively, and review the first several days of output by hand before trusting auto-publish. Start with YouTube privacy set to **unlisted** and Instagram auto-publish **off** until you are satisfied with what the pipeline produces.

One known limitation

Vision captions, hashtags and YouTube tags currently resolve the **primary** brand's vision chain regardless of which brand is publishing. This affects multi-brand setups only; a single-brand deployment is unaffected.